UNIT IV
THREADS AND GENERIC CLASSES

Understanding Threads, Needs of Multi-Threaded Programming, Thread Life-Cycle, Thread Priorities, Synchronizing Threads, Inter Communication of Threads, Critical Factor in Thread – Deadlock – Generic: Introduction to Generics – Built-in Generics collections – writing simple generic class.

4.1 Understanding Threads

4.1.1 Thread

A thread is a:

- Facility to allow multiple activities within a single process
- Referred as lightweight process
- A thread is a series of executed statements
- Each thread has its own program counter, stack and local variables
- A thread is a nested sequence of method calls
- Its shares memory, files and per-process state

4.1.2 Needs of Multi-Threaded Programming

Multitasking allows the user to handle multiple tasks together. In java, we can write programs to perform multitasking using multithreading concept. Thus java allows having multiple control flows in a single program by means of multithreading.

1. ***Improved Performance and Responsiveness***

Multi-threading allows programs to perform multiple operations or tasks simultaneously, leading to better CPU utilization and improved application performance, especially on multi-core processors.

2. ***Efficient Resource Utilization***

Threads share the same memory space and resources of a process, reducing the overhead of creating and managing multiple processes.

3. ***Responsiveness in User Interfaces***

In GUI-based applications, multi-threading ensures that long-running tasks do not block the main thread, keeping the user interface responsive.

4. ***Simplified Program Structure for Concurrent Tasks***

Tasks like I/O operations, network communication, or background computations can be organized more logically and efficiently using threads.

5. ***Better System Resource Management***

Threads are lighter than processes, allowing better scaling of applications and efficient use of system resources.

6. ***Real-Time and Asynchronous Processing***

Multi-threading enables real-time data processing and supports asynchronous task execution, essential for modern applications like servers, games, and simulations.

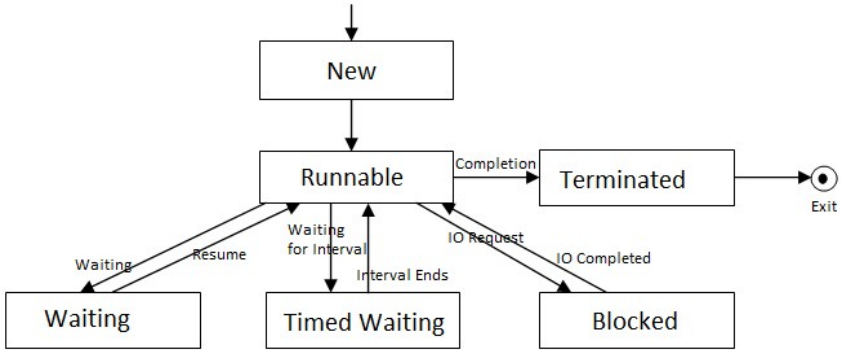
7. ***Parallelism***

Threads help achieve parallelism, allowing parts of a program to run in parallel, thereby speeding up computational tasks significantly.

Process	**Thread**
A program in execution	a subset(part) of the process
referred as task	referred as lightweight process
has its own address space	uses the process's address space and share it with the other threads of that process

Multithreading	**Multitasking**
Thread is the fundamental unit	Program is the fundamental unit
Multiple parts of a single program gets executed	Multiple program gets executed
Processor switches between multiple threads of the program	Processor switches between multiple programs or processor
Cost effective because CPU can be shared among multiple threads at a time	Expensive because when a particular process uses CPU, other process have to wait
Highly efficient	Less efficient
Helps in developing efficient application program	Helps in developing efficient OS programs

4.1.3 Thread Life-Cycle



New − A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a born thread.
Runnable − After a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.
Waiting – A thread waits for another thread to perform a task. Thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.
Timed Waiting − A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state, transitions back to the runnable state when that time interval expires.
Blocked – When a thread issues an input/output request, the operating system blocks the thread from executing until that I/O request completes.
Terminated (Dead) − A runnable thread enters the terminated state when it completes its task.

4.1.4 Creating a thread

Thread implementation in java can be achieved in two ways:

- Extending the java.lang.Thread class
- Implementing the java.lang.Runnable Interface

1) By extending Thread class

- The class should extend Java Thread class.
- The class should override the run() method.
- The functionality that is expected by the Thread to be executed is written in the run() method.

void start(): Creates a new thread and makes it runnable.
void run(): The new thread begins its life inside this method.

Example:

```
public class MyThread extends Thread {
public void run(){
System.out.println("thread is running...");
}
public static void main(String[] args) {
MyThread obj = new MyThread();
obj.start();
}}
```

Output

```
thread is running...
```

2) By Implementing Runnable interface

- The class should implement the Runnable interface
- The class should implement the run() method in the Runnable interface
- The functionality that is expected by the Thread to be executed is put in the run() method

Example:

```
public class MyThread implements Runnable {
public void run(){
System.out.println("thread is running..");
}
public static void main(String[] args) {
MyThread obj= new MyThread();
```

```
Thread t = new Thread(obj);
t.start();
}}
```

Output

thread is running...

4.1.5 Creating multiple threads

Multiple threads can be created both by extending Thread class and by implementing Runnable interface

1) By extending Thread class

Example:

```
class MyThread1 extends Thread {
public void run(){
System.out.println("First thread is running...");
} }
class MyThread2 extends Thread {
public void run(){
System.out.println("Second thread is running...");
} }
class Main{
public static void main(String[] args) {
MyThread1 obj1 = new MyThread1();
MyThread2 obj2 = new MyThread2();
obj1.start();
obj2.start();
}}
```

Output

```
First thread is running...
Second thread is running...
```

2) By Implementing Runnable interface

Example:

```
class MyThread1 implements Runnable {
public void run(){
System.out.println("First thread is running...");
} }
class MyThread2 implements Runnable {
public void run(){
System.out.println("Second thread is running...");
} }
class Main{
public static void main(String[] args) {
MyThread1 obj1 = new MyThread1();
MyThread2 obj2 = new MyThread2();
Thread T1=new Thread(obj1);
Thread T2=new Thread(obj2);
T1.start();
T2.start();
}}
```

Output

```
First thread is running...
Second thread is running...
```

4.1.6 Thread Priorities

Thread priority is an integer value to a thread that determines the relative importance of threads and helps the thread scheduler decide the order in which threads should be executed.

The integer value constant ranges from 1 to 10 and the constant defined can help to implement which are mentioned below.

Constant	Description
public static int NORM_PRIORITY	Sets the default priority for the Thread. (Priority: 5)
public static int MIN_PRIORITY	Sets the Minimum Priority for the Thread. (Priority: 1)
public static int MAX_PRIORITY	Sets the Maximum Priority for the Thread. (Priority: 10)

Setting and Getting Thread Priorities

Thread priorities can be set using:

thread.setPriority(int priority);

the priority of the given thread is returned using:

thread.getPriority();

Example:

```
import java.lang.*;
class Thread1 extends Thread {
   public void run()
   {
      System.out.println(Thread.currentThread().getName()+" is running with priority "
+ Thread.currentThread().getPriority());
   }
   public static void main(String[] args)
   {
      Thread1 t1 = new Thread1();
      Thread1 t2 = new Thread1();
      Thread1 t3 = new Thread1();
      // Display the priority of above threads
      System.out.println("t1 thread priority: " + t1.getPriority());
      System.out.println("t2 thread priority: " + t2.getPriority());
      System.out.println("t3 thread priority: " + t3.getPriority());
      t1.setPriority(2);
      t2.setPriority(5);
      t3.setPriority(8);
      System.out.println("t1 thread priority: " + t1.getPriority());
      System.out.println("t2 thread priority: " + t2.getPriority());
      System.out.println("t3 thread priority: " + t3.getPriority());
      t1.start();
      t2.start();
      t3.start();
   }
}
```

Output:

```
t1 thread priority: 5
t2 thread priority: 5
t3 thread priority: 5
t1 thread priority: 2
t2 thread priority: 5
t3 thread priority: 8
Thread-1 is running with priority 5
Thread-2 is running with priority 8
Thread-0 is running with priority 2
```

4.1.7 Synchronization

When we start two or more threads within a program, there may be a situation when multiple threads try to access the same resource and finally they can produce unforeseen result due to concurrency issues.

So there is a need to synchronize the action of multiple threads and make sure that only one thread can access the resource at a given point in time. This is implemented using a concept called monitors. Each object in Java is associated with a monitor, which a thread can lock or unlock. Only one thread at a time may hold a lock on a monitor.

Thread synchronization can be achieved:

1. Using synchronized method
2. Using synchronized block

1) Using synchronized method

```
class First{
synchronized public void display(String msg){
System.out.print(msg);
try{
Thread.sleep(1000);
}
catch(InterruptedException e){
System.out.print(e);
}}}
class Second extends Thread{
String msg;
First fobj;
Second(First fp,String str){
fobj = fp;
msg = str;
start();
}
public void run(){
fobj.display(msg);
}}
public class MyThread{
public static void main (String[] args){
First fnew = new First();
Second ss = new Second(fnew, "welcome");
Second ss1= new Second(fnew,"new");
Second ss2 = new Second(fnew, "programmer");
}}
```

Output

```
welcome
programmer
new
```

2) Using synchronized block

```
class First{
public void display(String msg){
synchronized(this){
System.out.println(msg);
try{
Thread.sleep(1000);
}
catch(InterruptedException e){
System.out.print(e);
}}}}
class Second extends Thread{
String msg;
First fobj;
Second(First fp,String str){
fobj = fp;
msg = str;
```

```
start();
}
public void run(){
fobj.display(msg);
}}
public class MyThread{
public static void main (String[] args){
First fnew = new First();
Second ss = new Second(fnew, "welcome");
Second ss1= new Second(fnew,"new");
Second ss2 = new Second(fnew, "programmer");
}}
```

Output

```
welcome
programmer
new
```

4.1.8 Inter-thread communication

Inter-thread communication or Co-operation is all about allowing synchronized threads to communicate with each other.

Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. It is implemented by following methods of Object class:

- wait()
- notify()
- notifyAll()

1) wait() method

Causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

2) notify() method

Wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation.

3) notifyAll() method

Wakes up all threads that are waiting on this object's monitor.

Example

```
class Customer{
int amount=10000;
synchronized void withdraw(int amount){
System.out.println("going to withdraw...");
if(this.amount<amount){
System.out.println("Less balance; waiting for deposit...");
try{wait();}catch(Exception e){}
}
this.amount-=amount;
System.out.println("withdraw completed...");
}
synchronized void deposit(int amount){
System.out.println("going to deposit...");
this.amount+=amount;
System.out.println("deposit completed... ");
notify();
} }
class Test{
public static void main(String args[]){
final Customer c=new Customer();
new Thread(){
```

```
public void run(){c.withdraw(15000);}
}.start();
new Thread(){
public void run(){c.deposit(10000);}
}.start();
}}
```

Output

```
going to withdraw...
Less balance; waiting for deposit...
going to deposit...
deposit completed...
withdraw completed...
```

4.1.9 Suspending – Resuming and stopping Threads

Stopping a Thread:

- A thread can be stopped from running further by issuing the following statement
  th.stop();
- By this statement, a thread enters a dead state
- From stopping state a thread can never return to a runnable state.

Blocking a thread:

- A thread can be temporarily stopped from running.
- Blocking or suspending of a thread
- Ways to block a thread

i. sleep() - blocked for some specific time and resumes to runnable state when time gets elapsed
ii. suspend() - blocked until further request [resume()] comes.
iii. wait() - suspended for some specific conditions and resumes to runnable state when notify() method is called.
iv. resume() - resume a thread which was suspended only using suspend() method.
v. notify() - wakes up only one thread waiting on the object and that thread starts execution even if there are multiple threads waiting for an object
vi. notifyall() - wakes up all threads waiting on the object and all thread start execution

Example:

```
class MyThread implements Runnable {
public void run() {
try {
Thread.sleep(500);
System.out.println(Thread.currentThread().getName());
} catch (Exception e) {
}
}
}
public class Main {
public static void main(String[] args) {
Thread t1 = new Thread(new MyThread());
Thread t2 = new Thread(new MyThread());
Thread t3 = new Thread(new MyThread());
t1.start();
t2.start();
t2.suspend();
t3.start();
t2.resume();
}
}
```

Output:

```
Thread-0
Thread-1
Thread-2
```

4.1.10 Critical Factor in Thread – Deadlock

Deadlock is a critical issue in multi-threaded programming where two or more threads are blocked forever, each waiting for the other to release a resource.

Causes of Deadlock:

1. **Mutual Exclusion** – At least one resource must be held in a non-shareable mode.
2. **Hold and Wait** – A thread holding one resource is waiting to acquire additional resources held by other threads.
3. **No Preemption** – Resources cannot be forcibly taken from threads holding them.
4. **Circular Wait** – A circular chain of threads exists where each thread waits for a resource held by the next thread in the chain.

```
class MyThread extends Thread {
   static final MyThread thread1 = new MyThread();
   static final MyThread thread2 = new MyThread();
   public synchronized void method1() {
      System.out.println(Thread.currentThread().getName() + " is in method1");
      try { Thread.sleep(100); } catch (Exception e) {}
      thread2.method2(); // tries to call method2 of thread2
   }
   public synchronized void method2() {
      System.out.println(Thread.currentThread().getName() + " is in method2");
      try { Thread.sleep(100); } catch (Exception e) {}
      thread1.method1(); // tries to call method1 of thread1
   }
   public void run() {
      if (Thread.currentThread().getName().equals("Thread-1")) {
         method1();
      } else {
         method2();
      }
   }
   public static void main(String[] args) {
      Thread t1 = new Thread(thread1, "Thread-1");
      Thread t2 = new Thread(thread2, "Thread-2");
      t1.start();
      t2.start();
   }
}
```

- Thread-1 runs method1() and tries to call thread2.method2() (but it's locked by Thread-2)
- Thread-2 runs method2() and tries to call thread1.method1() (but it's locked by Thread-1)

How to Avoid Deadlock in Java?

Deadlocks cannot be completely resolved. But we can avoid them by following basic rules mentioned below:

1. **Avoid Nested Locks**: Prevent threads from acquiring multiple locks simultaneously.
2. **Use Timeout**: Use try Lock() with a timeout.
3. **Lock Ordering**: Always acquire locks in a specific global order.
4. **Deadlock Detection**: Monitor for deadlock situations and recover.

4.2 Generics

Generic is a mechanism for creating a general model in which generic methods and generic classes enable programmers to specify a single method or a set of related methods and single class or a set of related classes for performing the desired task

Need for generic

- Using Generics, it is possible to create classes that work with different data types.
- It allows code reusability

- Compact code can be created

4.2.1 Built-in Generic Collections

Java provides several generic classes in the java.util package:

Collection Type	Description
ArrayList<T>	Resizable array
LinkedList<T>	Doubly-linked list
HashSet<T>	Set backed by a hash table
HashMap<K, V>	Map with key-value pairs
Stack<T>	LIFO stack
Queue<T>	FIFO queue

4.2.2 Generic methods

Generic methods allow programmers to write a generalized method for the methods of different data type.

Example

```
public class Demo{
public static<T extends Comparable<T>> void display(T[] a){
for(int i=0;i<5;i++)
System.out.print(a[i]+"  ");
}
public static void main(String args[]){
Float dbl_a[]={1.1f,2.2f,3.3f,4.4f,5.5f};
Integer int_a[]={1,2,3,4,5};
String str_a[]={"AA","BB","CC","DD","EE"};
System.out.print("Float elements are: ");
display(dbl_a);
System.out.print("\nInteger elements are: ");
display(int_a);
System.out.print("\nString elements are: ");
display(str_a);
}
}
```

Output

```
Float elements are: 1.1  2.2  3.3  4.4  5.5
Integer elements are: 1  2  3  4  5
String elements are: AA  BB  CC  DD  EE
```

4.2.3 Generic Classes

A generic class contains one or more generic data type

Example

```
public class Demo<T extends Comparable<T>>{
Demo(T[] a){
for(int i=0;i<5;i++)
System.out.print(a[i]+"  ");
}
public static void main(String args[]){
Float dbl_a[]={1.1f,2.2f,3.3f,4.4f,5.5f};
Integer int_a[]={1,2,3,4,5};
String str_a[]={"AA","BB","CC","DD","EE"};
System.out.print("Float elements are: ");
```

```
Demo<Float> dbl_D=new Demo<Float>(dbl_a);
System.out.print("\nInteger elements are: ");
Demo<Integer> int_D=new Demo<Integer>(int_a);
System.out.print("\nString elements are: ");
Demo<String> str_D=new Demo<String>(str_a);
}
}
```

Output

```
Float elements are: 1.1  2.2  3.3  4.4  5.5
Integer elements are: 1  2  3  4  5
String elements are: AA  BB  CC  DD  EE
```

4.2.4 Bounded Types

While creating objects to generic classes, we can pass any derived type as type parameters. Manytimes, it will be useful to limit the types thatcanbe passed to type parameters.for that purpose, bounded types are introduced in generics.

Using bounded types, we can make the objects of generic class to have data of specific derived type.

Example

```
public class Demo<T extends Number>{
Demo(T[] a){
for(int i=0;i<5;i++)
System.out.print(a[i]+"  ");
}
public static void main(String args[]){
Float dbl_a[]={1.1f,2.2f,3.3f,4.4f,5.5f};
Integer int_a[]={1,2,3,4,5};
String str_a[]={"AA","BB","CC","DD","EE"};
System.out.print("Float elements are: ");
Demo<Float> dbl_D=new Demo<Float>(dbl_a);
System.out.print("\nInteger elements are: ");
Demo<Integer> int_D=new Demo<Integer>(int_a);
//System.out.print("\nString elements are: ");
//Demo<String> str_D=new Demo<String>(str_a);
}
}
```

Output

```
Float elements are: 1.1  2.2  3.3  4.4  5.5
Integer elements are: 1  2  3  4  5
```

4.2.5 Restrictions and limitations

1. Generic types are compile time entities. The runtime execution is possible only if it is used along with raw type.
2. Primitive type parameters are not allowed for generic programming.
3. For the instance of generic class, throw and catch instances are not allowed.
4. Instantiation of generic parameter T is not allowed
   Eg. new T(); new t[]; // Error
5. Arrays of parameterized types are not allowed
   Eg. new stack<String>[10]; // Error
6. Static fields and static methods with type parameters are not allowed